CSIS 4495 – Applied Research Project

Mid-Course Progress Report

MedGuard: A Zero-Trust IoMT Health Data Pipeline for Real-Time Anomaly Detection and Automated Device Isolation

Item	Details
Team Lead (joint submissions)	Jude Enuanwa
Team members	Jude Enuanwa – Student ID 300403859 Oghenefejiro Agwere – Student ID 300395874
Course and section	CSIS 4495 – Section 002
Institution	Douglas College – Post-Baccalaureate Diploma in Emerging Technology (CIS)
Report date	October 2026
Demo video (public link)	CSIS4495_MdCourseDemoVideo.mp4
GitHub repository	https://github.com/Jud-e/F26_4495_S1_JudeE

1. Introduction

1.1 Domain and Background

The Internet of Medical Things (IoMT) is the growing network of connected medical devices, such as infusion pumps, cardiac monitors, pulse oximeters and wearable sensors, that collect and transmit physiological data for continuous clinical monitoring. Hospitals and remote-care programs adopt IoMT because it supports earlier detection of deterioration, reduces manual charting and enables data-driven treatment. The same connectivity creates a security surface that traditional hospital IT was not designed to defend.

Many IoMT devices have limited processing power, memory and battery capacity, and many are legacy hardware that cannot be patched or reconfigured. Heavyweight encryption or endpoint agents are often impossible to deploy on the device itself. The stakes are also unusually high: a compromised infusion pump or a spoofed cardiac monitor reading is not only a data breach, it is a potential patient-safety incident.

1.2 Problem Framing

This project asks: can a lightweight, real-time monitoring pipeline reliably detect anomalous or malicious activity in IoMT telemetry using unsupervised machine learning, while enforcing zero-trust access principles at a gateway, without requiring any change to the medical devices themselves?

The question matters for two reasons. First, because devices often cannot be modified, any practical defence has to sit at the network or gateway layer. Second, new and zero-day attack patterns are common, so signature-based detection is insufficient. Unsupervised anomaly detection learns what normal behaviour looks like and flags deviations, even when the specific attack has never been seen.

1.3 Related Work and Knowledge Gaps

Si-Ahmed, Al-Garadi and Boustia (2022) survey machine-learning intrusion detection for IoMT and observe that conventional cryptographic protections are hard to apply to medical equipment because of heterogeneous protocols and constrained computation, storage and energy, and that they are largely ineffective against novel attacks. They position ML-based detection as a complementary layer suited to these constraints. Reji, Pranggono, Marchang and Shenfield (2023) evaluate an ML anomaly detector for IoMT on a public dataset, reflecting the wider practice of validating IoMT detectors against benchmark traffic rather than purely synthetic data.

Work on ensemble anomaly detection with the CICIoMT2024 dataset (2025) notes that IoMT devices are especially exposed to denial-of-service, spoofing and reconnaissance attacks because of their minimal on-device processing, and argues that ML copes well with the high dimensionality of IoMT traffic. Research on zero-trust IoT environments observes that zero trust depends on continuous, risk-based access decisions, and that ML anomaly detection can supply the risk score in such a pipeline. For the detection method itself, Liu, Ting and Zhou (2008) introduced Isolation Forest, an unsupervised algorithm that isolates anomalies with random partitioning trees and is well suited to training on normal data only. Public benchmark data come from CICIDS2017 (Sharafaldin, Lashkari and Ghorbani, 2018) and the health-specific ECU-IoHT dataset (Ahmed et al., 2021).

A consistent gap runs through this literature. Most published work evaluates detection accuracy in isolation and pays little attention to how a detection result is operationalised in real time, meaning how a risk score becomes an alert or a device-isolation action in a working pipeline. This project does not propose a new detection algorithm. Its contribution is an applied, end-to-end system that connects zero-trust ingestion, unsupervised detection and an actionable alerting and isolation interface, at a scope appropriate for a two-person, one-semester build.

1.4 Hypotheses, Assumptions and Expected Benefit

- Unsupervised models (Isolation Forest, then Autoencoders) trained on normal telemetry will detect injected anomalous traffic with higher recall than a static rule-based threshold.
- A zero-trust gateway (authentication, schema validation, configuration checks) in front of the detection engine reduces malformed or unauthorised payloads reaching the model and improves signal quality.
- A 0 to 100 risk score is more actionable for a security analyst than a binary flag because it supports tiered responses (log, alert, isolate).
- Assumption: the project uses simulated devices and synthetic patient data only, so results demonstrate the design rather than clinical performance.

At this mid-course point, the gateway hypothesis can already be observed in the running system (Section 5.2). The detection hypotheses depend on the model work that is completed in the next phase, so no accuracy claims are made in this report.

2. Summary of the Initially Proposed Research Project

The proposal described MedGuard, a monitoring pipeline for simulated IoMT endpoints. The planned flow is: simulated devices post telemetry to a FastAPI gateway; the gateway authenticates, validates and checks each message, then publishes it to Kafka; a stream stage preprocesses it; an unsupervised model produces a 0 to 100 risk score; and the MedGuard dashboard shows alerts in real time with a one-click device isolation action.

Objectives and methodology

The project follows an applied, design-science approach: the main deliverable is a working system evaluated with standard intrusion-detection metrics. The four objectives are (1) a zero-trust ingestion gateway, (2) unsupervised anomaly detection evaluated on labelled attack traffic, (3) real-time scores and alerts on a dashboard, and (4) a one-click isolation action as the system's response.

Data and evaluation

Training and evaluation data come from the public CICIDS2017 and ECU-IoHT datasets, together with simulated device telemetry. Models are evaluated using precision, recall, F1-score, confusion matrices and ROC curves on held-out labelled attack classes. Utility is evaluated by simulating payload tampering and high-frequency traffic against the running pipeline and checking that the score rises, an alert is dispatched, and isolation halts further ingestion from the device.

Technology stack

Category	Technology
Platform	Docker and Docker Compose (Linux containers on Windows via WSL2)
Languages	Python (gateway, pipeline, ML); JavaScript (dashboard)
Frameworks	FastAPI, Kafka (kafka-python-ng client), Scikit-Learn / PyTorch
Database	PostgreSQL (device metadata, alerts, acknowledgments, verification logs)
Front end	React and TypeScript single-page app (Vite): admin console and nurse station, mocked role-based login
Back end	Zero-trust gateway, Kafka stream pipeline, WebSocket notifications
AI / LLM	One grounded summarisation call over logged events to draft incident reports

Extended ecosystem

The proposal refined the user model from a single analyst audience to two. A confirmed behavioural anomaly routes to the SOC dashboard with full technical detail and the isolation control. A structural device-configuration weakness, such as auto-lock disabled or default credentials unchanged, is detected at the gateway and routed to the assigned nurse as a plain-language remediation tip. Both channels require acknowledgment, and for configuration alerts the system verifies the fix by checking that later telemetry shows the secure setting.

Regulatory context

The design is informed by FDA Section 524B and the FDA premarket cybersecurity guidance (2023), IEC 81001-5-1 and IEC 62443. Because only synthetic data is used, PIPEDA and BC PIPA do not apply to the prototype, and the final report will state what a production system handling real health data would additionally require.

3. Changes to the Proposal

The following changes were made since the proposal. Each lists the original plan, the current plan and the justification.

Change	Original	Now	Justification
Patient context added to the checks	No patient data in the gateway checks.	A small patient registry maps each simulated device to a patient and a prescribed infusion rate. The gateway flags an infusion rate that differs from the prescription by more than 15 mL/hr.	A rate can be plausible in general and still wrong for a specific patient, so patient context makes the checks clinically meaningful. It was deliberately kept small: patient context informs checks that already exist, and no second ingestion system for full clinical records was built.
Hand-authored patient registry instead of Synthea	Generate FHIR/HL7-shaped records with Synthea.	Five hand-written patients (P-1001 to P-1005) matched to the five simulated devices. Synthea remains an optional stretch.	Synthea generates far more data than the pipeline consumes and needs a separate Java toolchain. A small registry gives deterministic, testable mismatches and keeps scope in check.
Gateway authentication gap identified	Token authentication and schema validation.	Per-device identity (per-device tokens or mTLS) added to the plan for Weeks 8 to 10.	The current bearer token is shared and device_id is self-reported, so one authenticated client could claim to be any device. A zero-trust design needs identity per device.
Training data for the live model	Train on CICIDS2017 and ECU-IoHT.	Train the live model on benign traffic generated by the simulator. Use CICIDS2017 and ECU-IoHT as a separate offline benchmark.	CICIDS2017 has more than 70 flow-level columns that do not match the fields the simulator sends (packet size, request rate, vitals, device type), so a model trained on it cannot score live messages. The public datasets still provide benchmark results.
Dashboard technology and access model	React Native, Flutter or Dash (to be decided). Roles for SOC and nurse.	A dashboard API service (FastAPI, Kafka consumer, WebSocket) serving a React and TypeScript web app with mocked sign-in. An admin console (all devices, audit log) and a nurse station (only assigned devices, desktop notifications).	A web app runs on any hospital workstation and needs no app-store release. Authentication is mocked for the prototype so the role rules (who sees and is notified about which device) can be demonstrated before a real identity provider is added.
Operations tooling	Manual start of each component.	Stream consumer containerised in Docker Compose and a one-command run script.	Reduces setup errors and makes the demo repeatable for both team members.
Responsibilities rebalanced	David led the nurse view and report agent.	David focuses on preprocessing and the models. Jude adds the dashboard API, nurse view and persistence. The report agent is shared.	Keeps the critical path (data pipeline and model) with one owner while the dashboard work proceeds in parallel.
Timeline	Preprocessing in Week 4, model in Week 5, penetration testing in Weeks 8 to 9.	Preprocessing completed Oct 14, model integration targeted Oct 22, penetration testing moved to Nov 13 to 25.	The pipeline work finished later than planned, so later phases shift by about one to two weeks. The final deadlines do not change.

4. Project Planning and Timeline

4.1 Responsibilities

Jude Enuanwa (Team Lead): repository and Docker infrastructure; zero-trust gateway (authentication, schema validation, configuration and prescription checks); device simulator and ground-truth logging; patient context; dataset loaders; dashboard API; React admin console and nurse station with mocked role-based sign-in, push notifications, acknowledgement and an audit log; device isolation (recorded in the dashboard now, gateway enforcement planned); verify-the-fix workflow; per-device authentication; joint submissions.

Oghenefejiro Agwere: TLS on the gateway; stream preprocessing; Isolation Forest training, evaluation and integration into the stream; Autoencoder and model tuning; precision, recall and ROC evaluation.

Shared: demo video, penetration testing, report agent, regulatory mapping, final report and presentation.

4.2 Updated Timeline and Milestones (Oct 21 to Dec 9)

Week	Dates	Focus	Deliverables and owner
6	Oct 17 – 23	Model and dashboard integration	Model saved and scoring in the stream (David); dashboard API, React admin and nurse views with mocked sign-in (Jude, done Oct 25); one-command run and README (Jude)
7	Oct 24 – 30	Mid-course demo	Rehearsal and screenshots; demo video (both); report and working implementation due Oct 26; in-class check-in
8	Oct 31 – Nov 6	Verify-the-fix and Autoencoder	Verify-the-fix workflow for configuration alerts (Jude); Autoencoder and tuning (David)
9	Nov 7 – 20	Isolation, identity and evaluation	Postgres persistence and gateway isolation enforcement (Jude); per-device authentication (Jude); precision, recall, ROC (David); Progress Report 2 due Nov 16
10	Nov 21 – 27	Security testing and reporting	Penetration testing (both); report agent (both); regulatory mapping starts; Progress Report 3 due Nov 23
11	Nov 28 – Dec 4	Polish and final check-in	Regulatory mapping complete; end-to-end polish; final report draft; Progress Report 4 due Nov 30; final check-in Dec 2
12	Dec 5 – 9	Final deliverables	Final report and code due Dec 7; final presentation and live demo Dec 9

4.3 Project Management Chart

Figure 1 is a Gantt chart of completed, in-progress and planned work by owner. A 50-item task tracker (spreadsheet) lists each task with its status, owner and date and mirrors this chart.

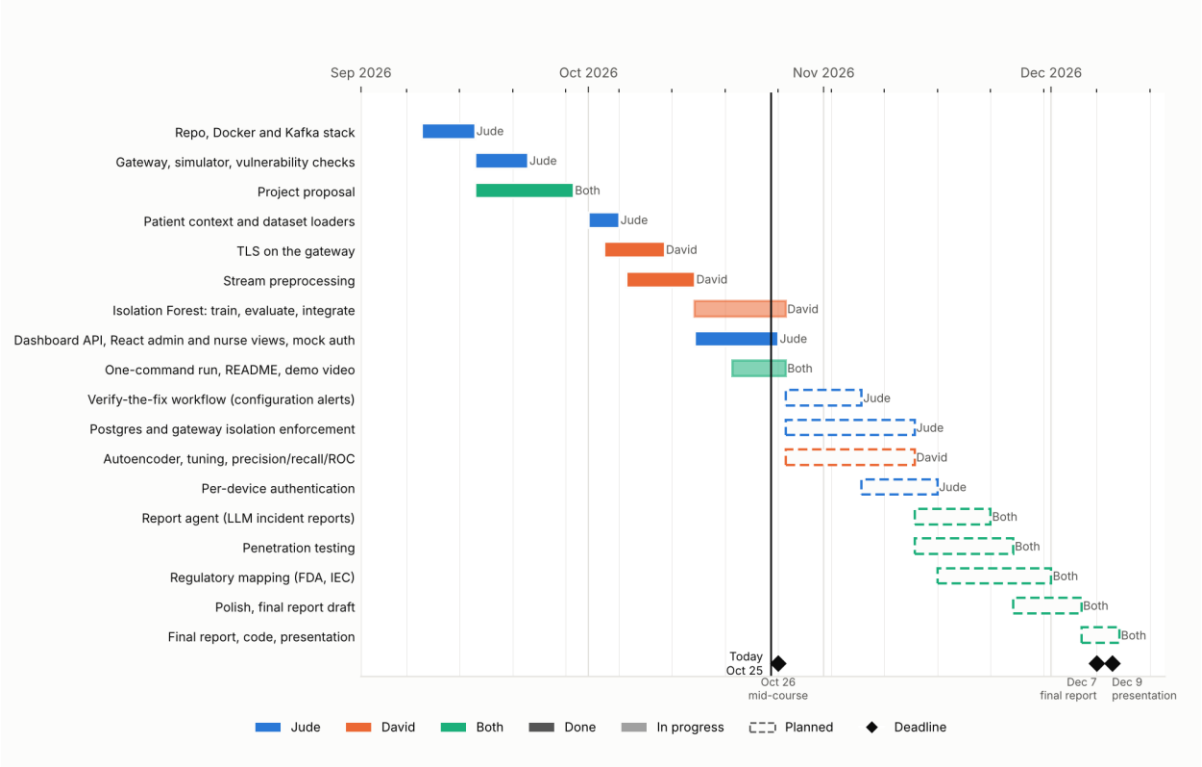


Figure 1. Gantt chart: completed, in-progress and planned tasks by owner, as of Oct 25, 2026.

5. Implemented Features

This section describes what each team member implemented. Screenshots and code excerpts marked in red must be added from the running system.

5.1 System Architecture

Figure 2 shows the intended finished architecture. Borders show status: solid is built, dotted is in progress and dashed is planned. Services communicate only through Kafka topics, so a slow or stopped stage builds a queue instead of losing data. The Postgres service is defined in the Compose file but is not yet used.

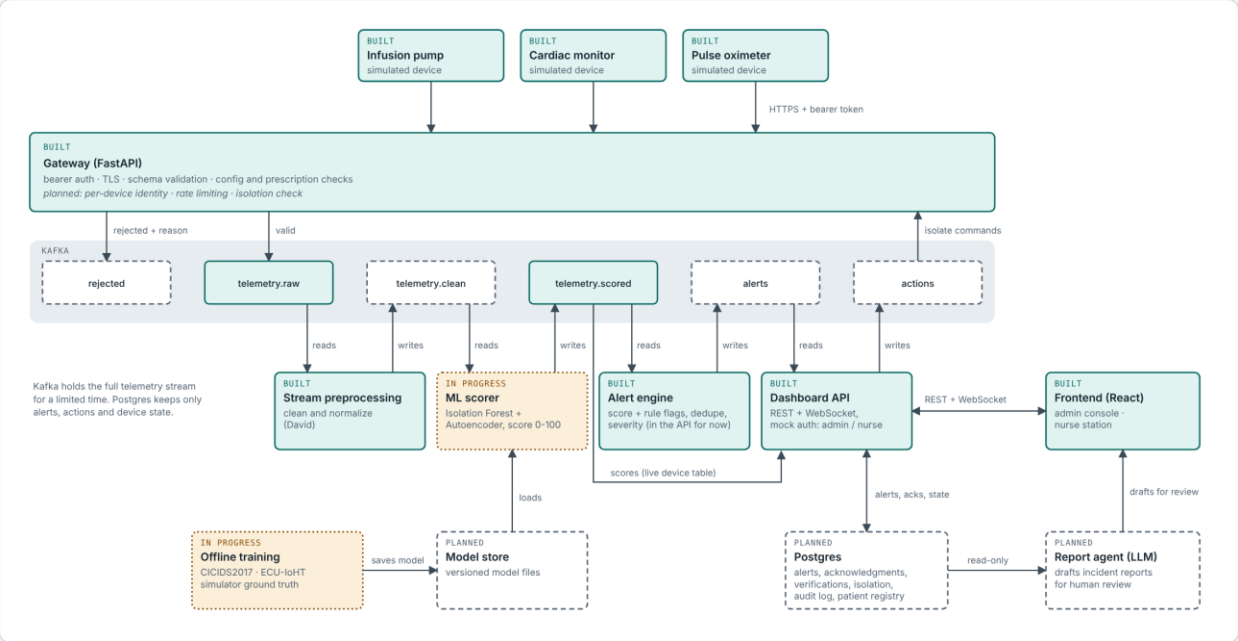


Figure 2. MedGuard architecture with build status per component.

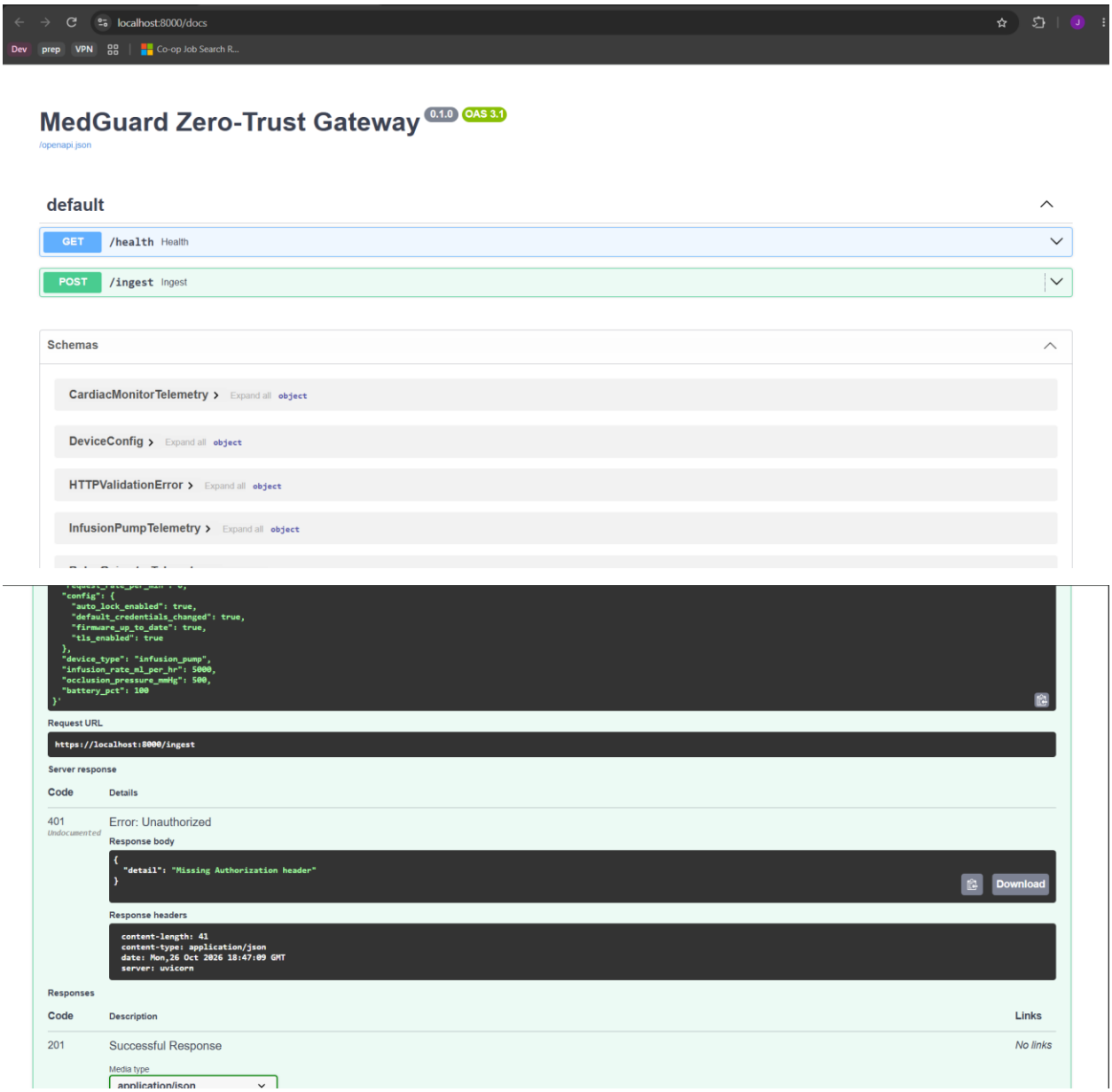
5.2 Jude: Zero-Trust Gateway

The gateway is a FastAPI service with a single ingestion endpoint, POST /ingest. Three independent defences run before a message reaches Kafka, and the first two are enforced automatically by FastAPI:

- **Authentication.** A dependency checks the Authorization: Bearer header against a secret from the environment. A missing or wrong token returns 401 and nothing is parsed.
- **Schema validation.** Pydantic models define one schema per device type (infusion pump, cardiac monitor, pulse oximeter) as a discriminated union on device_type. Numeric bounds are enforced and unknown fields are forbidden, so a tampered payload returns 422.
- **Configuration and prescription checks.** After a message is trusted, rules flag auto-lock disabled, default credentials unchanged, firmware outdated and TLS disabled. A separate check flags an infusion rate that differs from the patient's prescription by more than 15 mL/hr. Flags and plain-language remediation tips are attached to the message.

Valid messages are published to the iomt.telemetry.raw topic, keyed by device_id so one device's messages stay ordered. TLS is enabled by running uvicorn with a certificate and key generated with

mkcert. David wrote the TLS change; Jude merged it with the patient-context work and, when the SSL flags were found to have been dropped in a merge conflict resolution, traced the loss through the commit history, restored them (issue #4) and documented certificate setup in the README.

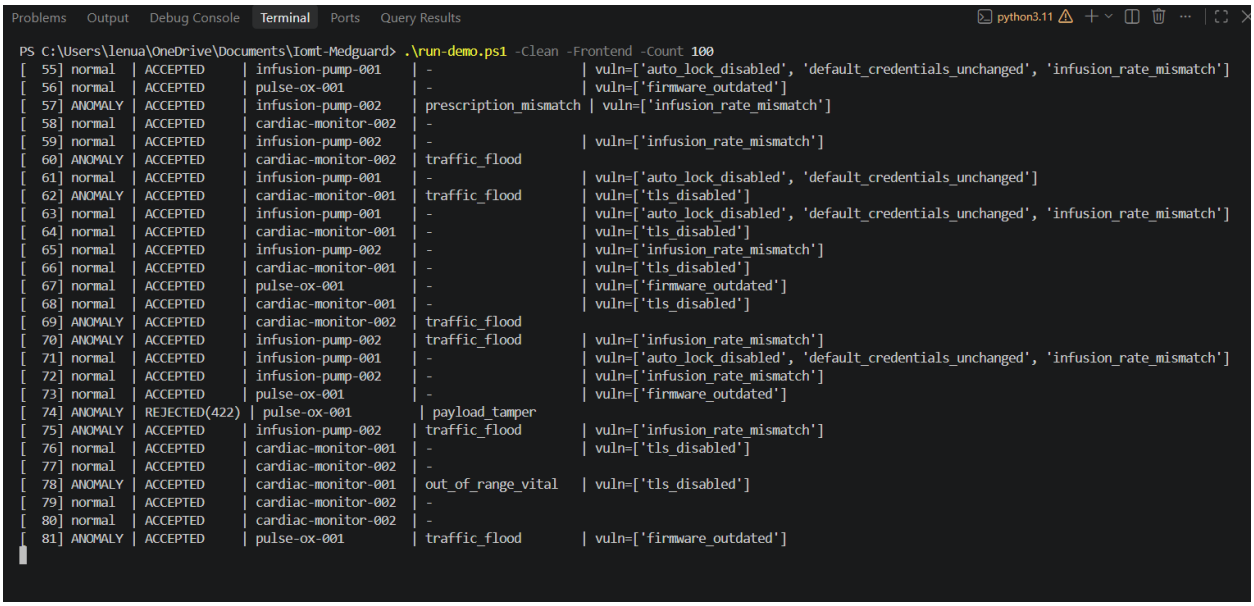


Known limitation. The token is shared and device_id is self-reported, so the gateway cannot yet prove which device sent a message. Per-device identity is planned (Section 4).

5.3 Jude: Device Simulator and Ground-Truth Logging

The simulator generates telemetry for five devices and posts it through the gateway over HTTPS, so the zero-trust layer is exercised instead of bypassed. Three devices carry deliberate configuration weaknesses so that vulnerability flags appear in normal operation. A configurable fraction of messages is replaced with an injected attack: out-of-range vitals, a traffic flood, payload tampering (expected to be rejected with 422) and, for pumps, a prescription mismatch. Every message is recorded in a ground-

truth CSV (message ID, label, attack type and whether rejection was expected), which later lets the model be scored against known labels.



5.4 Jude: Patient Context and Datasets

A small registry maps each device to a patient and, for pumps, a prescribed rate. The gateway loads it at startup from a read-only mounted file. For model data, a loader reads CICIDS2017 (2,827,876 flow records after cleaning), normalises column names, removes infinite and missing values, and splits benign records (for training) from all records with an is_attack column (for evaluation). Isolation Forest is unsupervised, so training uses benign data only. The label counts show strong class imbalance, so the headline metric will be a binary attack-versus-benign result with per-class results reported as a caveat.

5.5 Jude: Dashboard API

The dashboard API is a FastAPI service that consumes iomt.telemetry.scored in a background thread, keeps the latest state of every device in memory and pushes updates to browsers over a WebSocket. It derives alerts from each device's set of flags and risk score: a new alert is created only when the set of problems changes, so a device that keeps reporting the same flag does not flood the screen, and a problem that clears and later returns alerts again. Alerts are classed as high severity for an infusion-rate mismatch or a high model score, and medium otherwise.

Access is role-based and enforced on the server, not only in the interface. A sign-in endpoint issues a session token from a small set of mocked accounts (one admin, two nurses). Every REST call and the WebSocket require that token. Each nurse account has a list of device patterns, and the server filters the device list, the alert list and the live events so a nurse only receives data for assigned devices. Endpoints cover the device and alert lists, acknowledgement of an alert, isolation and restoration of a device (admin only) and the audit log and account list (admin only). Every sign-in, failed sign-in, sign-out, alert raised, acknowledgement and isolation is written to an audit log with the user, role, target and detail; for each alert the log also records which nurses were notified.

Known limitations. The accounts and passwords are hard-coded and the sessions, alerts and audit log are held in memory, so they reset when the service restarts. Isolating a device is recorded and audited, but the gateway does not yet block the device. Postgres persistence, real identity (per-device certificates and single sign-on) and gateway enforcement are planned.



5.6 Jude: React Frontend (Admin Console and Nurse Station)

The front end is a React and TypeScript single-page app built with Vite and served by the dashboard API container (a two-stage Docker build compiles it). A shared hook keeps a WebSocket connection with automatic reconnect and updates the device, alert and audit state as events arrive. A route guard sends signed-out users to the login page and sends each role to its own page.

- **Login.** Username and password against the mocked accounts, with demo accounts listed on the page so the roles can be shown quickly (Figure 3).
- **Admin console.** Five tabs: an overview with summary tiles, the highest-risk devices and the latest alerts; a device table with Isolate and Restore buttons; all alerts with who acknowledged them; a filterable audit log (by action, user and search text); and an access tab listing each account, its device patterns and whether it is online (Figures 4 and 6).
- **Nurse station.** Shows only the nurse's assigned devices as cards with plain-language status, flags rewritten for clinical staff, and a "what to do" tip. New alerts for those devices appear as an in-page toast and as a browser desktop notification once the nurse enables notifications. Each alert has an Acknowledge button, and acknowledged alerts move to their own list (Figure 5).

The interface follows the operating system's light or dark setting.

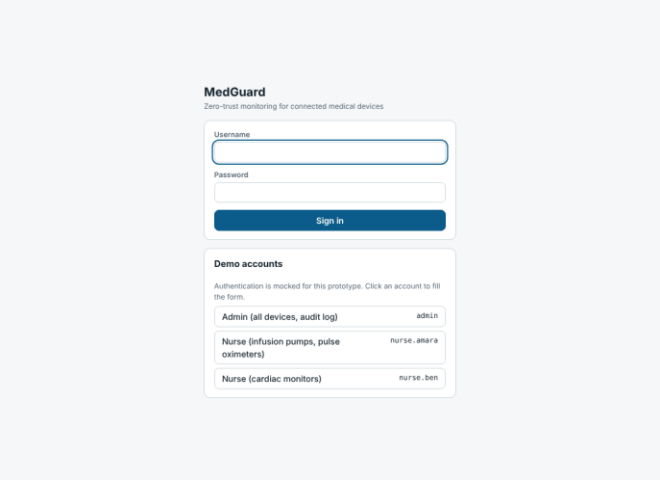


Figure 3. Login page with mocked demo accounts (sample data).

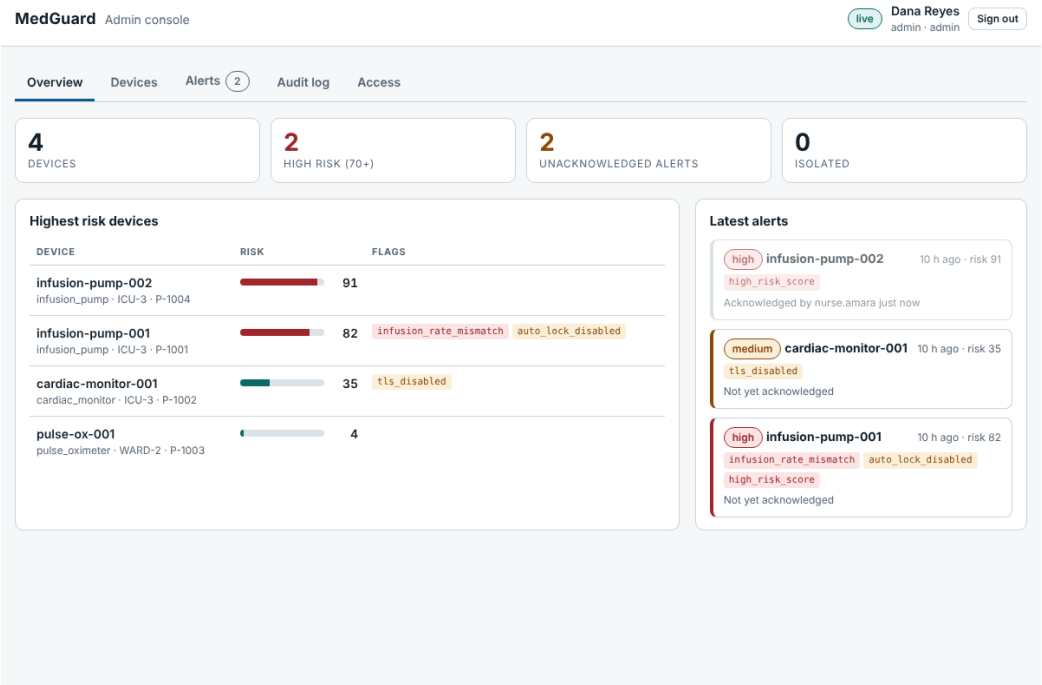


Figure 4. Admin console overview (sample data).

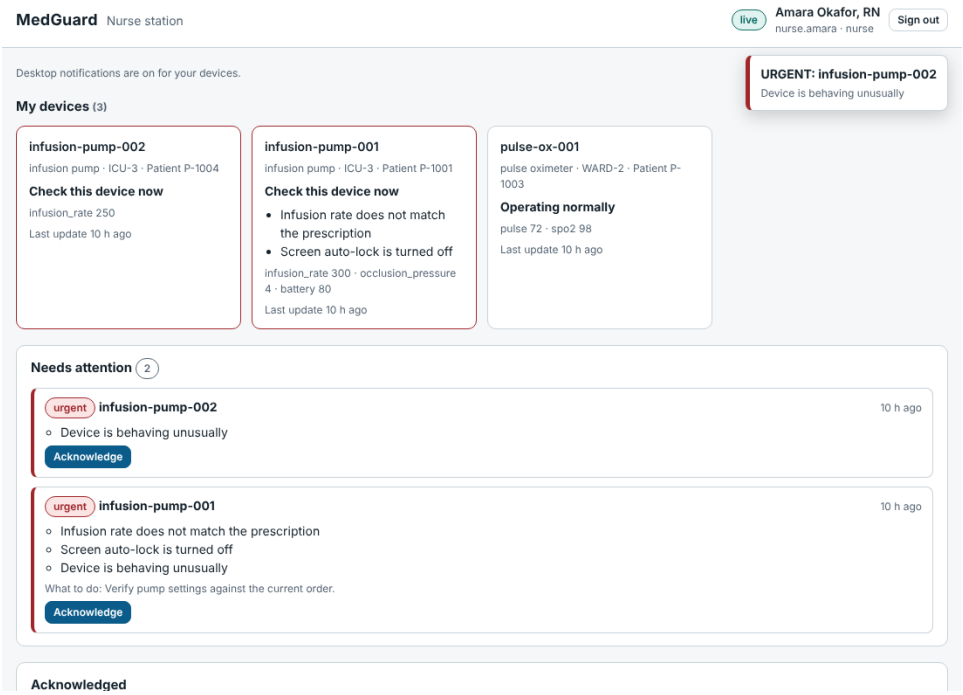


Figure 5. Nurse station with a notification toast for an assigned device (sample data).

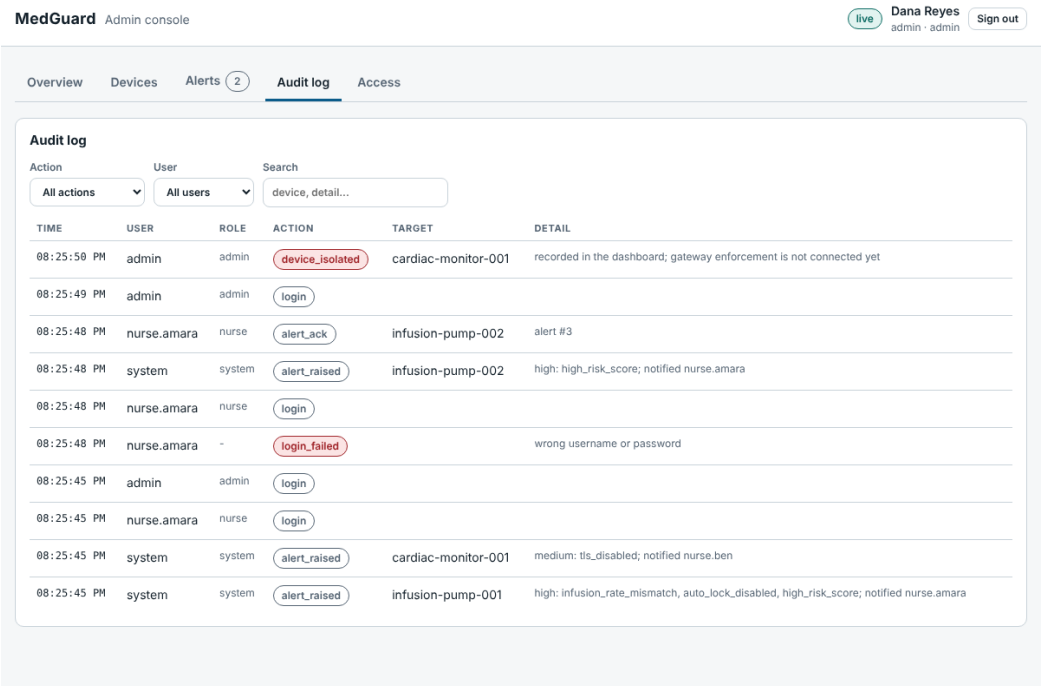


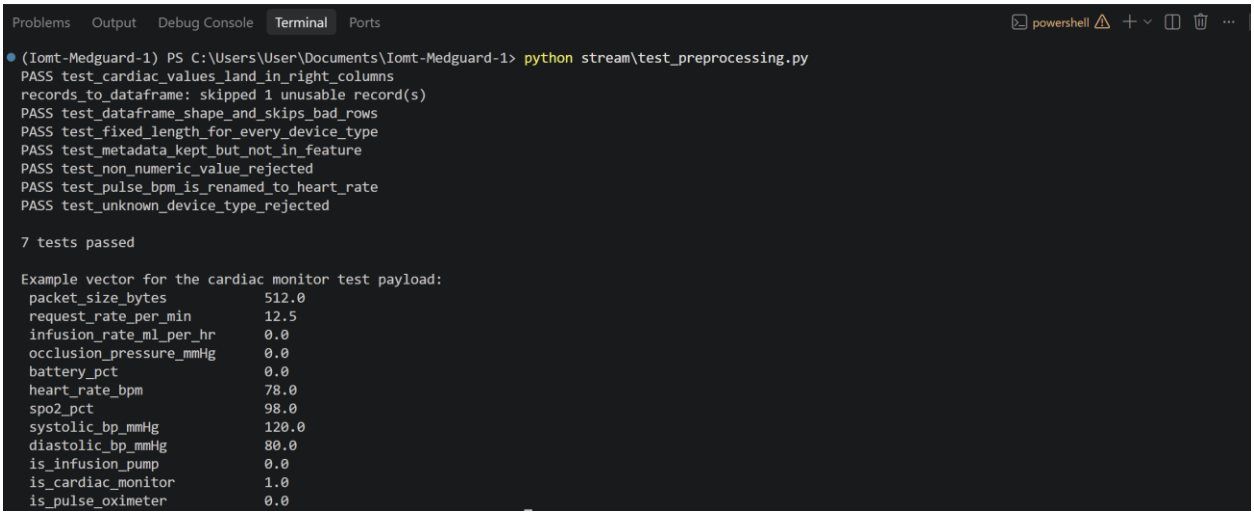
Figure 6. Admin audit log showing sign-ins, a failed sign-in, an alert, an acknowledgement and an isolation (sample data).

5.7 David: Stream Preprocessing

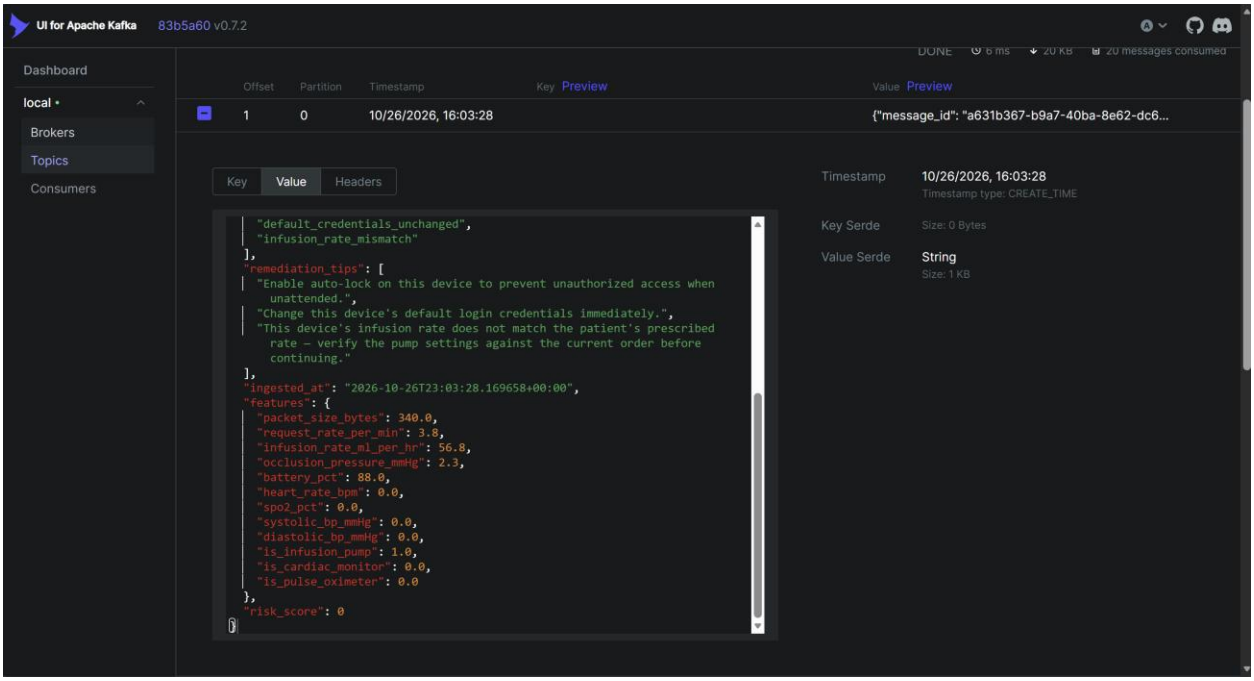
The preprocessing module converts one raw telemetry message into a fixed-length numeric feature vector, in a fixed column order, so the model always sees identically shaped data. It maps device-specific names onto one canonical name (the pulse oximeter's pulse_bpm becomes heart_rate_bpm), produces

nine numeric features (packet size, request rate, infusion rate, occlusion pressure, battery, heart rate, SpO2 and two blood-pressure values) and one-hot encodes the three device types. Missing values become zero. A record that cannot be vectorised raises a specific error and is skipped without stopping the stream. The module has no Kafka dependency, so the same function can serve training, live scoring and unit tests; seven unit tests pass. The stream consumer reads the raw topic, applies the function, and publishes the original message plus its features and a risk score to the scored topic.

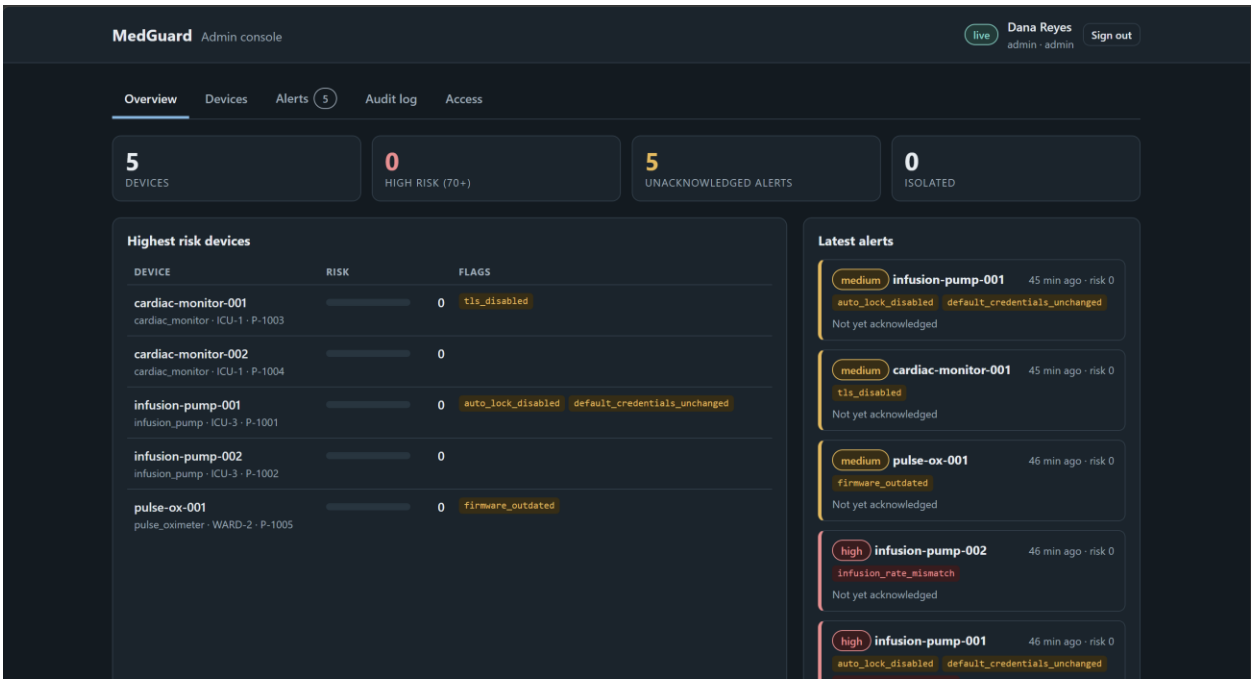
Test Passing



scored message in Kafka UI



5.8 David: Isolation Forest



5.9 Testing and Verification

Gateway behaviour was verified with automated tests for authentication rejection, schema rejection and correct pass-through, and by observing messages in Kafka UI. The preprocessing module has seven unit tests. The dashboard API has an automated test file (test_dash.py) that checks sign-in and sign-out, rejection of bad or missing tokens, nurse-versus-admin filtering of devices, alerts and WebSocket events, alert de-duplication and re-alerting, acknowledgement permissions, isolation and the audit entries. End-to-end behaviour (simulator, gateway, Kafka, consumer, dashboard) is checked by running the stack and the simulator together.

5.10 How to Run the Demo

The README describes installation and a one-command start. In short: install Docker Desktop (and make sure it is running), Python 3.12 or newer and mkcert; create the local certificate once with mkcert; install the simulator's Python dependencies; then run .\run-demo.ps1 from the project root. The script starts the containers, waits for the gateway, opens the dashboard at http://localhost:8100 and runs the device simulator. Sign in with one of the demo accounts (admin, or a nurse). Running it with -Clean first starts from empty data.

6. AI Use

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition (in your own words)
Claude (Claude Code)	Sonnet 5.5, Free plan	Docker Compose and Kafka setup help; debugging Docker virtualization and a Kafka client incompatibility on Python 3.12	I had to do some research into Kafka stream preprocessing as well as docker containerized env
Claude (Claude Code)	Sonnet 5.5, Free plan	Drafting the FastAPI gateway, schemas, authentication, vulnerability and prescription checks, and the device simulator	Schemas for the gateway had to emulate current medical device traffic as well as meet certain medical standards which I had to research.
Claude (Claude Code)	Sonnet 5.5, Free plan	Drafting the patient registry, dataset loader, dashboard API, React admin and nurse frontend with mocked authentication and audit log, and the one-command run script	Changed the front-end setup from one index page to mock a whole dashboard. This was to kind of stretch the capabilities of the project.
Claude (Claude Code)	Sonnet 5.5, Free plan	Diagnosing the lost TLS flags after a branch merge; drafting the proposal, task split, architecture diagram, Gantt chart and this report	Majority of this was fixing merge conflicts that arose after the first branch merge.

7. Work Date/Hours Logs

Entries record work as it was completed, with one entry per working day. Hours reflect actual time spent.

Student: Jude Enuanwa

Date	Hours	Description of work done
Sep 13, 2026	1	Created the MedGuard GitHub repo and wrote the initial README
Sep 15, 2026	2	Set up Docker Compose (Zookeeper, Kafka, Kafka UI) and tested the container pipeline; added setup instructions for teammate
Sep 21, 2026	1	Wrote the IoMT device simulator and the initial FastAPI gateway /ingest endpoint
Sep 22, 2026	0.5	Updated README with run instructions
Sep 23, 2026	2	Wrote the project proposal and checked it into DocumentsAndReports
Oct 1, 2026	1	Rewrote the README to cover the full stack and run steps

Date	Hours	Description of work done
Oct 2, 2026	2	Added patient context (registry, prescription checks); added training dataset loader and instructions (CICIDS2017, ECU-IoHT)
Oct 10, 2026	2	Merged teammate's TLS branch with patient-context, found the TLS flags dropped in the merge (issue #4), restored them, generated mkcert certs, wrote TLS setup docs, cleaned up .gitignore
Oct 11-12, 2026	2	Tested the full stack over HTTPS end to end and prepared the demo for Progress Report 1
Oct 14-15, 2026	1.5	Merged stream-preprocessing branch; built the dashboard API and live page
Oct 16-21, 2026	1	Containerised the stream consumer; wrote the run script; drafted this report.
Oct 22-25, 2026	2	Built the React admin console and nurse station with mocked sign-in, notifications, acknowledgement and audit log; added Docker build for the frontend.

Student: Oghenefejiro Agwere

Date	Hours	Description of work done
Sep 17, 2026	2	Researched the project's core technology stack and architecture to prepare for implementation: reviewed how Docker/Docker Compose, Kafka, Zookeeper, and Postgres function together in the pipeline; studied FastAPI/uvicorn's role as the zero-trust gateway and the mechanics of mTLS/token-based zero-trust auth.
Sep 21, 2026	1.5	Set up local development environment: installed Docker Desktop, cloned the MedGuard repository, and resolved a docker-compose.yml not found error by confirming working directory. Ran docker compose up -d --build to pull and build the Kafka/Zookeeper/Postgres/Kafka-UI images and the gateway service; verified all five containers (zookeeper, kafka, kafka-ui, gateway, iomt-db) reporting Up via docker compose ps. Confirmed gateway health endpoint, Kafka UI, and Postgres connectivity locally.
Sep 29, 2026	1	Added TLS to the gateway. Downloaded mkcert, created a certificate and private key, and changed the Docker Compose command so the gateway runs over HTTPS. Committed on the feature/tls-encryption branch.
Oct 14, 2026	2	Wrote the stream preprocessing module, which turns each raw message into 12 numbers, and the seven unit tests for it. Updated the stream consumer and checked a scored message in Kafka UI. Pushed the feature/stream-preprocessing branch and opened a pull request.
Oct 22, 2026	3	Pulled Jude’s latest code and ran his dataset loader on CICIDS2017 (2.8 million flow records) Wrote the risk-scoring code, which converts the Isolation Forest score into a 0 to 100 risk score with an alert line at 70.
Oct 26, 2026	1	Trained and evaluated the Isolation Forest on CICIDS2017. Committed the work and opened a pull request for feature/isolation-forest.

Date	Hours	Description of work done

8. Closing and References

The team acknowledges Douglas College CSIS 4495 course materials and instructor guidance, which shaped the structure and requirements of this project.

References

Ahmed, M., Byreddy, S., Nutakki, A., Sikos, L. F., & Haskell-Dowland, P. (2021). ECU-IoHT: A dataset for analyzing cyberattacks in Internet of health things. Ad Hoc Networks, 122, 102621. https://doi.org/10.1016/j.adhoc.2021.102621

Costa, G., Forestiero, A., Macrì, D., & Ortale, R. Trustworthy precision medicine: An interpretable approach to detecting anomalous behavior of IoT devices. IOS Press. https://ebooks.iospress.nl/pdf/doi/10.3233/SHTI240068

Enhanced anomaly detection in IoMT networks using ensemble AI models on the CICIoMT2024 dataset. (2025). arXiv preprint arXiv:2502.11854. https://arxiv.org/pdf/2502.11854

Hsiao, H.-C., Tsai, C.-P., Li, Z.-X., Lee, C.-H., Chen, J.-S., Lai, Y.-C., Wang, J.-C., Li, S.-C., Gao, J.-C., & Lee, Y.-H. (2023). Load balancing algorithms and their impacts on Apache Kafka. In 2023 IEEE International Conference on Big Data (pp. 1726–1735). IEEE.

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In 2008 Eighth IEEE International Conference on Data Mining (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17

Reji, A., Pranggono, B., Marchang, J., & Shenfield, A. (2023). Anomaly detection for the internet-of-medical-things. In 2023 IEEE International Conference on Communications Workshops (ICC Workshops) (pp. 1944–1949). IEEE. https://shura.shu.ac.uk/31706

Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), Volume 1 (pp. 108–116). https://doi.org/10.5220/0006639801080116

Si-Ahmed, A., Al-Garadi, M. A., & Boustia, N. (2022). Survey of machine learning based intrusion detection methods for Internet of Medical Things. arXiv preprint arXiv:2202.09657. https://arxiv.org/pdf/2202.09657

U.S. Food and Drug Administration. (2023). Cybersecurity in medical devices: Quality system considerations and content of premarket submissions (final guidance). https://www.fda.gov/regulatory-information/search-fda-guidance-documents

Advanced anomaly detection and threat intelligence in zero trust IoT environments using machine learning. arXiv preprint arXiv:2604.23332. https://arxiv.org/pdf/2604.23332

Appendix A: Prompt History

Prompts sent to Claude during this project, in order, with the date sent. Long pasted logs are truncated.

[99] (2026-10-10) python ingestion/simulate_devices.py --count 20

Posting to https://localhost:8000/ingest (anomaly chance: 5%). Ground truth logged to ground_truth_log.csv. Ctrl+C to stop.
[1] normal | CONN-ERROR | infusion-pump-002 | -
[2] normal | CONN-ERROR | cardiac-monitor-002 | -
[3] normal | CONN-ERROR | cardiac-monitor-001 | -
[4] normal | CONN-ERROR | infusion-pump-001 | -
[5] normal | CONN-ERROR | pulse-ox-001 | -

[100] (2026-10-10) docker compose logs gateway --tail 30

gateway | INFO: Will watch for changes in these directories: ['/app']
gateway | INFO: Uvicorn running on https://0.0.0.0:8000 (Press CTRL+C to quit)
gateway | INFO: Started reloader process [1] using WatchFiles
gateway | INFO: Started server process [8]
gateway | INFO: Waiting for application startup.
gateway | INFO: Application startup complete.
gateway | INFO: 172.18.0.1:58628 - "GET /docs HTTP/1.1" 200 OK
gateway | INFO: 172.18.0.1:58628 - "GET /openapi.json HTTP/1.1" 200 OK
$env:REQUESTS_CA_BUNDLE = "$(mkcert -CAROOT)\rootCA.pem"
python ingestion/simulate_devices.py --count 20
it worked after running this command

[101] (2026-10-10) yeah make a short readme

[102] (2026-10-10) What's next for me then, cuz david is working on preprocessing rn and i know his next task is the isolation forest model

[103] (2026-10-10) So dashboard that can be viewed is the next checkpoint

[104] (2026-10-10) for the progress report, a demo is necessary in a sense

[105] (2026-10-10) But what exactly would the small demo be showing? A risk score based on thje model?

[110] (2026-10-14) Alright what's left in project implementation?

[111] (2026-10-14) create a spreadsheet wih what's been done so far and what's left

[112] (2026-10-14) what's the next step?

[113] (2026-10-14) 0

[114] (2026-10-14) run me through the architecture real quick. i know there's a dashboard frontend, there's a backend, and this ml part is application logic, and kafka and docker are the environments

[115] (2026-10-14) so nothing gets stored essentially since there's no db

[116] (2026-10-14) i'm asking for an architecture for the intended finished product, not for the demo due 28th

[117] (2026-10-14) Yeah give it aproper diagram so i can reference it later on

[120] (2026-10-14) next step, the stub scorer, explain

[121] (2026-10-14) honestly, instead of having s stub scorer, let's just do the preprocessing and ml scorer

[127] (2026-10-15) Just for future purposes, is there a way to run al this inone line without having to type it all in diff terminals as well

[132] (2026-10-15) failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the path is correct and if the daemon is running: open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.

Waiting for the gateway...

[139] (2026-10-24) Can we open a separate chat for brainstorming that has the memories of this chat

[140] (2026-10-24) If we can't that's fine. I want to know what we can do. Not for the midterms of course but later on to make this project/app a product worthy application. Like how can we maximize its potential

[141] (2026-10-24) Well I see it as a startup idea. And I want to build it before December if possible. I want to be done with the school version by next two weeks and start this as soon as possible

[146] (2026-10-25) {"detail":"Frontend not built. Run `npm run build` in dashboard-api/frontend."}

[147] (2026-10-25) It already has alerts noted, I'm guessing from previous iterations ran in groundtruth

[148] (2026-10-25) Nah, I just want to know how to wipe it so give me those instructions. Also, do I need to rebuild this node environment anytime I make changes, if yes, can I just add the command to the runps1 file

[150] (2026-10-25) Okay, the front end looks good. I may change the colors later on. To a dark and lightmode. Since we've worked on this all this way, what do you think we can squeeze in before report is due